

Project on data re-uploading for a universal quantum classifier

Perez-Salinas et al., "Data re-uploading for a universal quantum classifier", Quantum 4, 226 (2020).

The goal of this project is to reproduce and extend selected results from the target paper using Qiskit. The project focuses on single-qubit and multi-qubit quantum classifiers based on data re-uploading, with and without entanglement. The quantum models will be compared with classical neural-network baselines. The final implementation will also be applied to the Iris dataset.

Questions

1. Can a single-qubit data re-uploading circuit learn nonlinear classification boundaries?
2. How does classification accuracy depend on the number of re-uploading layers?
3. Do multi-qubit classifiers reduce the circuit depth required to obtain good performance?
4. Does entanglement improve the performance of the multi-qubit classifier?
5. How do the quantum classifiers compare with classical neural networks?
6. Does the same approach work on a standard real dataset such as Iris?
7. Optional: how does a shallow trained classifier behave when evaluated on real quantum hardware or a noisy simulator?

Task 1 - Build the dataset module

- Implement synthetic binary classification datasets: circle, crown or annulus, squares/checkerboard-like data, and wavy or non-convex boundaries. ([see datasets provided in the paper](#))
- Each dataset must include training/test splits, a visualization function, and consistent label encoding.
- Implement Iris preprocessing using `sklearn.datasets.load_iris`.
- For Iris, include binary setosa vs versicolor and versicolor vs virginica tasks. Optional: full three-class one-vs-rest classification.
- Use standardization and angle rescaling for quantum circuits. PCA may be used for two-dimensional visualization.

Task 2 - Implement the single-qubit data re-uploading classifier

- Implement a one-qubit parametrized quantum classifier in Qiskit (look at the implementation on pennylane https://pennylane.ai/qml/demos/tutorial_data_reuploading_classifier)
- Use repeated data encoding layers followed by trainable rotations.
- Use an expectation value such as $\langle Z \rangle$ as the model output.
- Test several depths, for example $L = 1, 2, 4, 8, 12, 16$.
- Produce training/test accuracy versus depth, loss curves, decision-boundary plots, parameter counts, and discussion of initialization dependence.

Task 3 - Implement multi-qubit classifiers without entanglement

- Implement a multi-qubit version where each qubit receives data re-uploading layers but no entangling gates are used.
- Test at least $n = 2$ and $n = 3$ qubits with $L = 1, 2, 4, 8$ layers.
- Compare single-qubit and multi-qubit models at fixed depth and approximately fixed number of trainable parameters.

Task 4 - Implement multi-qubit classifiers with entanglement

- Add entangling gates between data re-uploading layers. A linear CNOT chain or ring topology is sufficient.
- Compare the same model with and without entanglement.

- Report train accuracy, test accuracy, loss curves, number of parameters, and qualitative differences in decision boundaries.

Task 5 - Classical baselines

- Implement classical baselines using scikit-learn.
- Train all classical models on the same train/test splits as the quantum models.
- Report train accuracy, test accuracy, parameter counts where meaningful, runtime, and decision boundaries for two-dimensional datasets.

Task 6 - Apply the models to Iris

- Apply the implemented quantum and classical classifiers to Iris.
- Required binary tasks: setosa vs versicolor and versicolor vs virginica.
- Optional: full three-class classification using one-vs-rest quantum classifiers.

Suggested repository structure

```
data-reuploading-classifier/
|
|-- README.md
|-- requirements.txt
|-- environment.yml
|-- .gitignore
|
|-- notebooks/
|   |-- 00_datasets.ipynb
|   |-- 01_single_qubit_classifier.ipynb
|   |-- 02_multiqubit_classifier.ipynb
|   |-- 03_classical_baselines.ipynb
|   |-- 04_results_summary.ipynb
|   |-- 05_iris_application.ipynb
|
|-- src/
|   |-- __init__.py
|   |-- datasets.py
|   |-- quantum_models.py
|   |-- classical_models.py
|   |-- plotting.py
```